

Competitive Programming Notebook

py.Sandu

Contents

1	DP	2
1.1	Knapsack	2
1.2	Coins Change	2
1.3	Longest Increasing Subsequence	2
1.4	Kadane Algorithm	2
1.5	Fenwick Tree	2
2	Geometry	2
2.1	Isleft	2
2.2	Winding Number	2
2.3	Convex Hull	3
2.4	Cross Product	3
2.5	Point	3
3	String	3
3.1	Kmp	3
4	Graph	4
4.1	Floyd-warshall	4
4.2	Dfs	4
4.3	Reverse Graph	4
4.4	Mst Prim	4
4.5	Ford-fulkerson	5
4.6	Eulerean Path	5
4.7	Dijkstra	5
4.8	Bellman-ford	5
4.9	Bfs	6
5	Math	6
5.1	Matrix Fibonacci	6
5.2	Extended Euclidean Algorithm	6
5.3	Gcd	6
5.4	Fft	6
5.5	Modular Exponentiation	7
5.6	Siege Of Eratosthenes	7
5.7	Matrix Multiplication	7
5.8	Solve Linear System	7
6	General	7
7	DS	7
7.1	Monotonic Stack	7
7.2	Trie	8
7.3	Union Find	9
7.4	Seg Tree	9

1 DP

1.1 Knapsack

```

1 // Knapsack problem implementation using dynamic
  programming
2 // Time complexity: O(n*w), where n is the number of
  items and w is the maximum weight
3 // Space complexity: O(n*w) for the dp table
4
5 int knapsack(int w, vector<ii>& items){
6     int n = items.size();
7     vector<vi> dp(n+1, vi(w+1, 0));
8
9     for(int i = 1 ; i <= n ; i++){
10        for(int j = 0 ; j <= w ; j++){
11            if(items[i-1].first > j) dp[i][j] = dp[i-1][j];
12            else dp[i][j] = max(dp[i-1][j], dp[i-1][j-
13            items[i-1].first] + items[i-1].second);
14        }
15    }
16    return dp[n][w];
17 }
```

1.2 Coins Change

```

1 // Find the minimum number of coins needed to make a
  given amount using a set of coin denominations
2 // Time complexity: O(n * m) where n is the amount
  and m is the number of coin denominations
3
4 int coinChange(const vector<int>& coins, int amount)
  {
5     vector<int> dp(amount + 1, INT_MAX);
6     dp[0] = 0;
7
8     for (int i = 1; i <= amount; i++) {
9         for (const int& coin : coins) {
10             if (coin <= i && dp[i - coin] != INT_MAX)
11                 dp[i] = min(dp[i], dp[i - coin] + 1);
12         }
13     }
14 }
15
16 return dp[amount] == INT_MAX ? -1 : dp[amount];
17 }
```

1.3 Longest Increasing Subsequence

```

1 // Find the length of the longest increasing
  subsequence in a given array
2 // Time complexity: O(n log n) where n is the length
  of the input array
3
4 int lis(vector<int>& arr) {
5     vector<int> dp;
6     for (int num : arr) {
7         auto it = lower_bound(dp.begin(), dp.end(),
8         num);
9         if (it == dp.end()) {
10             dp.push_back(num);
11         } else {
12             *it = num;
13         }
14     }
15     return dp.size();
16 }
```

1.4 Kadane Algorithm

```

1 // Find maximum subarray sum using Kadane's algorithm
2 // Time complexity: O(n) where n is the length of the
  input array
3
4 int maxSubArraySum(const vector<int>& arr) {
5     int maxSum = arr[0];
6     int maxLocal = arr[0];
7
8     for (int i = 0; i < arr.size(); i++) {
9         maxLocal = max(arr[i], maxLocal + arr[i]);
10        maxSum = max(maxSum, maxLocal);
11    }
12
13    return maxSum;
14 }
```

1.5 Fenwick Tree

```

1 // Fenwick Tree to efficiently calculate and update
  prefix sums of an array of values.
2 // Time complexity: O(log N) for queries and updates
3
4 int sum(int i) {
5     int sum = 0;
6     while(i > 0) {
7         sum += T[i];
8         i -= i & -i; // flip the last bit
9     }
10    return sum;
11 }
12
13 void add(int i, int k) {
14     while (i < T.size()) {
15         T[i] += k;
16         i += i & -i; // add last set bit
17     }
18 }
19
20 vi make(vi arr) {
21     vi tree = arr;
22
23     for(int i = 0 ; i < tree.size() ; i++) {
24         int p = i + (i & -i) // index to parent
25         if (p < t.size()) {
26             tree[p] = tree[p] + tree[i];
27         }
28     }
29     return tree;
30 }
```

2 Geometry

2.1 Isleft

```

1 // Check if a point is to the left of a segment
  defined by two points
2 bool isLeft(const Point& p, const Point& v1, const
  Point& v2) {
3     return (v2.x - v1.x) * (p.y - v1.y) - (v2.y - v1.
4     y) * (p.x - v1.x) > 0;
5 }
```

2.2 Winding Number

```

1 // Check if a point is inside a polygon using the
  winding number algorithm
2
3 bool isInsidePolygon(const Point& p, const vector<
  Point>& polygon) {
```

```

4   int windingNumber = 0;
5   int n = polygon.size();
6
7   for (int i = 0; i < n; i++) {
8       const Point& v1 = polygon[i];
9       const Point& v2 = polygon[(i + 1) % n];
10
11      if (v1.y <= p.y) {
12          if (v2.y > p.y && isLeft(p, v1, v2)) {
13              ++windingNumber;
14          }
15      } else {
16          if (v2.y <= p.y && !isLeft(p, v1, v2)) {
17              --windingNumber;
18          }
19      }
20  }
21
22  return windingNumber != 0;
23 }

```

2.3 Convex Hull

```

1 // Returns a list of points on the convex hull
2 // in counter-clockwise order
3 vector<Point> convex_hull(vector<Point> A)
4 {
5     int n = A.size(), k = 0;
6
7     if (n <= 3)
8         return A;
9
10    vector<Point> ans(2 * n);
11
12    // Sort points lexicographically
13    sort(A.begin(), A.end());
14
15    // Build lower hull
16    for (int i = 0; i < n; ++i) {
17
18        // If the point at K-1 position is not a part
19        // of hull as vector from ans[k-2] to ans[k
20        -1]
21        // and ans[k-2] to A[i] has a clockwise turn
22        while (
23            k >= 2
24            && cross_product(ans[k - 2], ans[k - 1],
25                A[i])
26                <= 0)
27            k--;
28        ans[k++] = A[i];
29    }
30
31    // Build upper hull
32    for (size_t i = n - 1, t = k + 1; i > 0; --i) {
33
34        // If the point at K-1 position is not a part
35        // of hull as vector from ans[k-2] to ans[k
36        -1]
37        // and ans[k-2] to A[i] has a clockwise turn
38        while (k >= t
39            && cross_product(ans[k - 2], ans[k -
40                1],
41                    A[i - 1])
42                    <= 0)
43            k--;
44        ans[k++] = A[i - 1];
45    }
46
47    // Resize the array to desired size
48    ans.resize(k - 1);
49
50    return ans;

```

```

47 }

```

2.4 Cross Product

```

1 double cross_product(Point O, Point A, Point B)
2 {
3     return (A.x - O.x) * (B.y - O.y)
4         - (A.y - O.y) * (B.x - O.x);
5 }

```

2.5 Point

```

1 struct Point {
2     double x, y;
3 };

```

3 String

3.1 Kmp

```

1 // Find all occurrences of a pattern in a given text
2 // using the Knuth-Morris-Pratt (KMP) algorithm
3 // Time complexity: O(n + m) where n is the length of
4 // the text and m is the length of the pattern
5
6 vector<int> LPS(const string& pattern) {
7     int m = pattern.size();
8     vector<int> lps(m, 0);
9     int len = 0; // length of the previous longest
10    prefix suffix
11    int i = 1;
12
13    while (i < m) {
14        if (pattern[i] == pattern[len]) {
15            len++;
16            lps[i] = len;
17            i++;
18        } else {
19            if (len != 0) {
20                len = lps[len - 1];
21            } else {
22                lps[i] = 0;
23                i++;
24            }
25        }
26    }
27    return lps;
28 }
29
30 vector<int> KMP(const string& text, const string&
31    pattern) {
32    vector<int> lps = LPS(pattern);
33    vector<int> occurrences;
34    int n = text.size();
35    int m = pattern.size();
36    int i = 0; // index for text
37    int j = 0; // index for pattern
38
39    while (i < n) {
40        if (text[i] == pattern[j]) {
41            i++;
42            j++;
43        }
44        if (j == m) {
45            occurrences.push_back(i - j); // Pattern
46            found at index (i - j)
47            j = lps[j - 1]; // Continue to search for
48            the next match
49        } else if (i < n && text[i] != pattern[j]) {
50            if (j != 0) {
51                j = lps[j - 1];
52            }
53        }
54    }
55    return occurrences;
56 }

```

```

46         } else {
47             i++;
48         }
49     }
50 }
51 return occurrences;
52 }

```

4 Graph

4.1 Floyd-warshall

```

1 // Find the shortest paths in a weighted graph with
  // positive or negative edge weights (but with no
  // negative cycles).
2 // Time Complexity:  $O(V^3)$ 
3 // Space Complexity:  $O(V^2)$ 
4
5 void floydWarshall(Graph& graph, vector<vector<int>>&
  dist) {
6     int V = graph.size();
7
8     for(int i = 0; i < V; i++){
9         for (int j = 0; j < V; j++) {
10             if (i == j) {
11                 dist[i][j] = 0;
12             } else if (dist[i][j] == 0) {
13                 dist[i][j] = INT_MAX; // No edge
14             }
15         }
16     }
17
18     for (int k = 0; k < V; k++) {
19         for (int i = 0; i < V; i++) {
20             for (int j = 0; j < V; j++) {
21                 if (dist[i][k] != INT_MAX && dist[k][j]
22                     != INT_MAX) {
23                     dist[i][j] = min(dist[i][j], dist
24                                     [i][k] + dist[k][j]);
25                 }
26             }
27         }
28     }
29 }

```

4.2 Dfs

```

1 // Depth First Search (DFS)
2 // Time Complexity:  $O(V + E)$ 
3 // Space Complexity:  $O(V)$ 
4
5 // Recursive DFS
6 void dfs(const Graph& graph, int vertex, vector<bool>
  & visited) {
7     visited[vertex] = true;
8
9     for (int neighbor : graph[vertex]) {
10         if (!visited[neighbor]) {
11             dfs(graph, neighbor, visited);
12         }
13     }
14 }
15
16 // Iterative DFS
17 void dfsIterative(const Graph& graph, int start) {
18     int V = graph.size();
19     vector<bool> visited(V, false);
20     stack<int> s;
21     s.push(start);
22
23     while (!s.empty()) {

```

```

24         int vertex = s.top();
25         s.pop();
26
27         if (!visited[vertex]) {
28             visited[vertex] = true;
29
30             for (int neighbor : graph[vertex]) {
31                 if (!visited[neighbor]) {
32                     s.push(neighbor);
33                 }
34             }
35         }
36     }

```

4.3 Reverse Graph

```

1 // Reverse the edges of a directed graph
2 // Time Complexity:  $O(V + E)$ 
3 // Space Complexity:  $O(V + E)$ 
4
5 Graph reverseGraph(const Graph& graph) {
6     int V = graph.size();
7     Graph reversedGraph(V);
8
9     for (int u = 0; u < V; u++) {
10         for (int v : graph[u]) {
11             reversedGraph[v].push_back(u);
12         }
13     }
14
15     return reversedGraph;
16 }

```

4.4 Mst Prim

```

1 // Prim's algorithm for finding the Minimum Spanning
  // Tree
2 // Time complexity:  $O(E \log V)$  where E is the number
  // of edges and V is the number of vertices
3
4 struct Edge {
5     int u, v, weight;
6     bool operator<(const Edge& other) const {
7         return weight > other.weight; // Min-heap
8     }
9 };
10
11 vector<Edge> primMST(const vector<vector<int>>& graph
  ) {
12     int n = graph.size();
13     vector<bool> visited(n, false);
14     vector<Edge> mst;
15     priority_queue<Edge> pq;
16
17     // Start with the first vertex
18     visited[0] = true;
19     for (int i = 0; i < n; ++i) {
20         if (graph[0][i] != 0) {
21             pq.push({0, i, graph[0][i]});
22         }
23     }
24
25     while (!pq.empty()) {
26         Edge current = pq.top();
27         pq.pop();
28
29         if (visited[current.v]) continue;
30
31         visited[current.v] = true;
32         mst.push_back(current);
33
34         for (int i = 0; i < n; ++i) {

```

```

35         if (!visited[i] && graph[current.v][i] != 0) {
36             pq.push({current.v, i, graph[current.v][i]});
37         }
38     }
39 }
40
41 return mst;
42 }

```

4.5 Ford-fulkerson

```

1 // Find maximum flow in graph
2 // Time complexity:  $O(|V| * E^2)$ 
3 // Space complexity:  $O(V)$ 
4 bool bfs(int residual[V][V], int start, int target,
5   vi parent){
6     vector<bool> visited(V, false);
7
8     queue<int> q;
9     q.push(start);
10    visited[start] = true;
11    parent[start] = -1;
12
13    while(!q.empty()){
14        int u = q.front();
15        q.pop();
16
17        for(int v = 0 ; v <= V ; v++){
18            if(!visited[v] and residual[u][v] > 0){
19                if(v == target){
20                    parent[v] = u;
21                    return true;
22                }
23                q.push(v);
24                parent[v] = u;
25                visited[v] = true;
26            }
27        }
28    }
29    return false;
30 }
31 int fordFulkerson(int g[V][V], int start, int target)
32 {
33     int u, v;
34     int residual[V][V];
35
36     for(u = 0 ; u < V ; u++)
37         for(v = 0 ; v < V ; v++)
38             residual[u][v] = g[u][v];
39
40     vi parent(V);
41     int maxFlow = 0;
42
43     while(bfs(residual, start, target, parent)){
44         int pathFlow = INT_MAX;
45         for(v = target ; v != start ; v = parent[v]){
46             u = parent[v];
47             pathFlow = min(pathFlow, residual[u][v]);
48         }
49
50         for(v = target ; v != start ; v = parent[v]){
51             u = parent[v];
52             residual[u][v] -= pathFlow;
53             residual[v][u] += pathFlow;
54         }
55         maxFlow += pathFlow;
56     }
57     return maxFlow;

```

4.6 Eulerean Path

```

1 // Find the Eulerian path in a graph using Hierholzer
2 // Time complexity:  $O(E)$  where E is the number of
3 // edges in the graph
4 void findEulerianPath(const vector<vector<int>>&
5   graph, int start, vector<int>& path) {
6     stack<int> s;
7     s.push(start);
8
9     while (!s.empty()) {
10        int v = s.top();
11        if (graph[v].empty()) {
12            path.push_back(v);
13            s.pop();
14        } else {
15            int u = graph[v].back();
16            graph[v].pop_back(); // Remove the edge
17            // from v to u
18            s.push(u);
19        }
20    }

```

4.7 Dijkstra

```

1 // Find shortest path from a source vertex to all
2 // other vertices in a graph with non-negative edge
3 // weights
4 // Time Complexity:  $O((V + E) \log V)$ 
5 // Space Complexity:  $O(V)$ 
6 void dijkstra(const Graph& graph, int src) {
7     int V = graph.size();
8     vector<int> dist(V, INT_MAX);
9     dist[src] = 0;
10
11     priority_queue<pair<int, int>, vector<pair<int,
12     int>>, greater<pair<int, int>>> pq; // Min-heap
13     priority queue
14     pq.push({0, src});
15
16     while (!pq.empty()) {
17         int u = pq.top().second;
18         pq.pop();
19
20         for (const auto& adj : graph[u]) {
21             int v = adj.first;
22             int weight = adj.second;
23
24             if (dist[u] != INT_MAX && dist[u] +
25             weight < dist[v]) {
26                 dist[v] = dist[u] + weight;
27                 pq.push({dist[v], v});
28             }
29         }
30     }

```

4.8 Bellman-ford

```

1 // Find the shortest path from a source vertex to all
2 // other vertices in a graph with negative weight
3 // edges and detect negative weight cycles.
4 // Time Complexity:  $O(V * E)$ 
5 // Space Complexity:  $O(V)$ 
6 void bellmanFord(Graph& graph, int src) {
7     int V = graph.size();
8     vector<int> dist(V, INT_MAX);

```

```

8     dist[src] = 0;
9
10    for (int i = 1; i < V; i++) {
11        for (int u = 0; u < V; u++) {
12            for (int v = 0; v < V; v++) {
13                if (graph[u][v] != 0 && dist[u] !=
14                INT_MAX && dist[u] + graph[u][v] < dist[v]) {
15                    dist[v] = dist[u] + graph[u][v];
16                }
17            }
18        }
19
20    // Check for negative weight cycles
21    for (int u = 0; u < V; u++) {
22        for (int v = 0; v < V; v++) {
23            if (graph[u][v] != 0 && dist[u] !=
24            INT_MAX && dist[u] + graph[u][v] < dist[v]) {
25                cout << "Graph contains negative
26                weight cycle" << endl;
27                return;
28            }
29        }
30    }

```

4.9 Bfs

```

1 // Breadth First Search (BFS)
2 // Time Complexity: O(V + E)
3 // Space Complexity: O(V)
4 void bfs(const Graph& graph, int start) {
5     int V = graph.size();
6     vector<bool> visited(V, false);
7     queue<int> q;
8
9     visited[start] = true;
10    q.push(start);
11
12    while (!q.empty()) {
13        int vertex = q.front();
14        q.pop();
15        cout << vertex << " ";
16
17        for (int neighbor : graph[vertex]) {
18            if (!visited[neighbor]) {
19                visited[neighbor] = true;
20                q.push(neighbor);
21            }
22        }
23    }
24 }

```

5 Math

5.1 Matrix Fibonacci

```

1 // Calculate the Nth Fibonacci number using matrix
2 // exponentiation.
3 // Time Complexity: O(log N)
4 // Space Complexity: O(1)
5 void matrixExponentiation(vector<vector<long long>>&
6 matrix, int power) {
7     int n = matrix.size();
8     vector<vector<long long>> result(n, vector<long
9     long>(n, 0));
10
11    // Initialize result as the identity matrix
12    for (int i = 0; i < n; i++) {
13        result[i][i] = 1;

```

```

12    }
13
14    while (power) {
15        if (power % 2 == 1) {
16            matrixMultiplication(result, matrix,
17            result);
18        }
19        matrixMultiplication(matrix, matrix, matrix);
20        power /= 2;
21    }
22    matrix = result;
23 }

```

5.2 Extended Euclidean Algorithm

```

1 // Extended Euclidean Algorithm
2 // Returns (gcd(a, b), x, y) such that a*x + b*y =
3 gcd(a, b)
4 int extendedEuclidean(int a, int b, int &x, int &y){
5     if(a == 0){
6         x = 0;
7         y = 1;
8         return b;
9     }
10
11    int x1, y1;
12    int gcd = extendedEuclidean(b % a, a, x1, y1);
13    x = y1;
14    y = x1 - (b / a) * y1;
15    return gcd;
16 }

```

5.3 Gcd

```

1 // Calculates the Greatest Common Divisor (GCD) of
2 // two integers using Euclidean algorithm
3 // Time Complexity: O(log(min(a, b)))
4 // Space Complexity: O(log(min(a, b)))
5 int gcd(int a, int b){
6     if (a == 0) return b;
7     return gcd(b % a, a);
8 }

```

5.4 Fft

```

1 // Fast Fourier Transform (FFT) implementation
2 // Time complexity: O(n log n)
3 using cd = complex<double>;
4 const double PI = acos(-1);
5
6 void fft(vector<cd>& a, bool invert) {
7     int n = a.size();
8     if (n == 1) return;
9
10    vector<cd> a0(n / 2), a1(n / 2);
11    for (int i = 0; i < n / 2; i++) {
12        a0[i] = a[2 * i];
13        a1[i] = a[2 * i + 1];
14    }
15    fft(a0, invert);
16    fft(a1, invert);
17
18    double ang = 2 * PI / n * (invert ? -1 : 1);
19    cd w1(1), wn(cos(ang), sin(ang));
20    for (int i = 0; i < n / 2; i++) {
21        a[i] = a0[i] + w * a1[i];
22        a[i + n / 2] = a0[i] - w * a1[i];
23        if (invert) {
24            a[i] /= 2;

```

```

25         a[i + n / 2] /= 2;
26     }
27     w *= wn;
28 }
29 }

```

5.5 Modular Exponentiation

```

1 // Calculates (x^n) % MOD
2 // Time Complexity: O(log n)
3 // Space Complexity: O(1)
4
5 int powMod(int x, int n, int MOD){
6     int res = 1;
7
8     while(n >= 1){
9         if(n & 1){
10             res = (res * x) % MOD;
11             n--;
12         }
13         else{
14             x = (x * x) % MOD;
15             n /= 2;
16         }
17     }
18     return res;
19 }

```

5.6 Sieve Of Eratosthenes

```

1 // Find all prime numbers up to a given limit using
  // the Sieve of Eratosthenes algorithm
2 // Time complexity: O(n log log n) where n is the
  // limit
3
4 vector<int> findPrimes(int limit) {
5     vector<bool> isPrime(limit + 1, true);
6     isPrime[0] = isPrime[1] = false;
7
8     for (int i = 2; i * i <= limit; i++) {
9         if (isPrime[i]) {
10             for (int j = i * i; j <= limit; j += i) {
11                 isPrime[j] = false;
12             }
13         }
14     }
15
16     vector<int> primes;
17     for (int i = 2; i <= limit; i++) {
18         if (isPrime[i]) {
19             primes.push_back(i);
20         }
21     }
22     return primes;
23 }

```

5.7 Matrix Multiplication

```

1 // Multiply two matrices A and B, and return the
  // result in matrix C.
2 // Time Complexity: O(n^3)
3 // Space Complexity: O(n^2)
4
5 vector<vector<int>> matrixMultiplication(const vector
  <vector<int>>& A, const vector<vector<int>>& B) {
6     int n = A.size();
7     vector<vector<int>> C(n, vector<int>(n, 0)); //
  Initialize result matrix with zeros
8
9     for (int i = 0; i < n; i++) {
10         for (int j = 0; j < n; j++) {
11             for (int k = 0; k < n; k++) {

```

```

12                 C[i][j] += A[i][k] * B[k][j];
13             }
14         }
15     }
16     return C;
17 }

```

5.8 Solve Linear System

```

1 // Solve a system of linear equations using LU
  // decomposition
2 // Time complexity: O(n^3) for decomposition, O(n^2)
  // for forward and backward substitution
3
4 void luDecomposition(const vector<vector<double>>& A,
  vector<vector<double>>& L, vector<vector<double>
  >>& U) {
5     int n = A.size();
6     for (int i = 0; i < n; i++) {
7         for (int j = 0; j < n; j++) {
8             if (i <= j) {
9                 U[i][j] = A[i][j];
10                 for (int k = 0; k < i; k++) {
11                     U[i][j] -= L[i][k] * U[k][j];
12                 }
13             } else {
14                 L[i][j] = A[i][j];
15                 for (int k = 0; k < j; k++) {
16                     L[i][j] -= L[i][k] * U[k][j];
17                 }
18                 L[i][j] /= U[j][j];
19             }
20         }
21     }
22 }
23
24 void forwardSubstitution(const vector<vector<double>
  >>& L, const vector<double>& b, vector<double>& y
  ) {
25     int n = L.size();
26     for (int i = 0; i < n; i++) {
27         y[i] = b[i];
28         for (int j = 0; j < i; j++) {
29             y[i] -= L[i][j] * y[j];
30         }
31     }
32 }
33
34 void backwardSubstitution(const vector<vector<double>
  >>& U, const vector<double>& y, vector<double>& x
  ) {
35     int n = U.size();
36     for (int i = n - 1; i >= 0; i--) {
37         x[i] = y[i];
38         for (int j = i + 1; j < n; j++) {
39             x[i] -= U[i][j] * x[j];
40         }
41         x[i] /= U[i][i];
42     }
43 }
44 }

```

6 General

7 DS

7.1 Monotonic Stack

```

1 // Increasing Monotonic Stack
2 // Time Complexity: O(n)

```

```

3 // Space Complexity: O(n)
4
5 vector<int> nextGreaterElements(vector<int>& arr) {
6     int n = arr.size();
7     vector<int> res(n, -1);
8     stack<int> st;
9
10    for (int i = 0; i < 2 * n; i++) {
11        int num = arr[i % n];
12        while (!st.empty() && arr[st.top()] < num) {
13            res[st.top()] = num;
14            st.pop();
15        }
16        if (i < n) {
17            st.push(i);
18        }
19    }
20
21    return res;
22 }
23
24 // Decreasing Monotonic Stack
25 // Time Complexity: O(n)
26 // Space Complexity: O(n)
27
28 vector<int> nextSmallerElements(vector<int>& arr) {
29     int n = arr.size();
30     vector<int> res(n, -1);
31     stack<int> st;
32
33     for (int i = 0; i < 2 * n; ++i) {
34         int num = arr[i % n];
35         while (!st.empty() && arr[st.top()] > num) {
36             res[st.top()] = num;
37             st.pop();
38         }
39         if (i < n) {
40             st.push(i);
41         }
42     }
43
44     return res;
45 }

```

7.2 Trie

```

1 // class for a node of the trie
2 class TrieNode {
3 public:
4     bool endofWord;
5     TrieNode* children[26];
6
7     // Constructore to initialize a trie node
8     TrieNode()
9     {
10         endofWord = false;
11         for (int i = 0; i < 26; i++) {
12             children[i] = nullptr;
13         }
14     }
15 };
16
17 // class for the Trie implementation
18 class Trie {
19 private:
20     TrieNode* root;
21
22 public:
23     Trie() { root = new TrieNode(); }
24
25     // Function to insert a word into the trie
26     void insert(string word)
27     {

```

```

28     TrieNode* node = root;
29     for (char c : word) {
30         int index = c - 'a';
31         if (!node->children[index]) {
32             node->children[index] = new TrieNode
33         };
34         node = node->children[index];
35     }
36     node->endofWord = true;
37 }
38
39 // Function to search a word in the trie
40 bool search(string word)
41 {
42     TrieNode* node = root;
43     for (char c : word) {
44         int index = c - 'a';
45         if (!node->children[index]) {
46             return false;
47         }
48         node = node->children[index];
49     }
50     return node->endofWord;
51 }
52
53 // Function to check if there is any word in the
54 // trie that starts with the given prefix
55 bool startsWith(string prefix)
56 {
57     TrieNode* node = root;
58     for (char c : prefix) {
59         int index = c - 'a';
60         if (!node->children[index]) {
61             return false;
62         }
63         node = node->children[index];
64     }
65     return true;
66 }
67
68 // Function to delete a word from the trie
69 void deleteWord(string word)
70 {
71     TrieNode* node = root;
72     for (char c : word) {
73         int index = c - 'a';
74         if (!node->children[index]) {
75             return;
76         }
77         node = node->children[index];
78     }
79     if (node->endofWord == true) {
80         node->endofWord = false;
81     }
82 }
83
84 // Function to print the trie
85 void print(TrieNode* node, string prefix) const
86 {
87     if (node->endofWord) {
88         cout << prefix << endl;
89     }
90     for (int i = 0; i < 26; i++) {
91         if (node->children[i]) {
92             print(node->children[i],
93                 prefix + char('a' + i));
94         }
95     }
96 }
97
98 // Function to start printing from the root

```

```

99     void print() const { print(root, ""); }
100 };

```

7.3 Union Find

```

1  int find(vector<int>& union_find, int a) {
2      vector<int> path;
3
4      while(union_find[a] >= 0){
5          path.push_back(a);
6          a = union_find[a];
7      }
8
9      for(int vertice : path)
10         union_find[vertice] = a;
11
12     return a;
13 }
14
15 void unite(vector<int>& union_find, int a, int b) {
16     a = find(union_find, a);
17     b = find(union_find, b);
18
19     if(a == b) return;
20
21     if(abs(union_find[a]) < abs(union_find[b])) swap(
22         a, b);
23
24     union_find[a] += union_find[b];
25     union_find[b] = a;
26 }
27 // Example usage for Hamming distance problem
28
29 class Solution {
30 public:
31     int minimumHammingDistance(vector<int>& source,
32         vector<int>& target, vector<vector<int>>&
33         allowedSwaps) {
34         vector<int> union_find(source.size(), -1);
35         unordered_map<int, unordered_map<int, int>>
36         count;
37
38         for(int i = 0; i < allowedSwaps.size(); i++){
39             int a = allowedSwaps[i][0];
40             int b = allowedSwaps[i][1];
41             unite(union_find, a, b);
42         }
43
44         for(int i = 0; i < source.size(); i++){
45             int group = find(union_find, i);
46             count[group][source[i]]++;
47             count[group][target[i]]--;
48         }
49
50         int ans = 0;
51         for(auto& pair : count)
52             for(auto& freq : pair.second)
53                 ans += abs(freq.second);
54
55         return ans/2;
56 }
57 };

```

7.4 Seg Tree

```

1  // Segment Tree implementation
2  // Time complexity: O(log n) for updates and queries
3

```

```

4  template<typename T>
5  class SegmentTree {
6  private:
7      int n;
8      vector<T> tree;
9      T combine(T a, T b) {
10         // Combine function for the segment tree (e.g
11         //., sum, min, max)
12         return a + b; // Example: sum
13     }
14     void build(vector<T>& arr, int v, int tl, int tr)
15     {
16         if (tl == tr) {
17             tree[v] = arr[tl];
18         } else {
19             int tm = (tl + tr) / 2;
20             build(arr, v * 2, tl, tm);
21             build(arr, v * 2 + 1, tm + 1, tr);
22             tree[v] = combine(tree[v * 2], tree[v * 2
23                 + 1]);
24         }
25     }
26     void update(int v, int tl, int tr, int pos, T
27         new_val) {
28         if (tl == tr) {
29             tree[v] = new_val;
30         } else {
31             int tm = (tl + tr) / 2;
32             if (pos <= tm) {
33                 update(v * 2, tl, tm, pos, new_val);
34             } else {
35                 update(v * 2 + 1, tm + 1, tr, pos,
36                     new_val);
37             }
38             tree[v] = combine(tree[v * 2], tree[v * 2
39                 + 1]);
40         }
41     }
42     T query(int v, int tl, int tr, int l, int r) {
43         if (l > r) {
44             return T(); // Return identity element (e
45             .g., 0 for sum)
46         }
47         if (l == tl && r == tr) {
48             return tree[v];
49         }
50         int tm = (tl + tr) / 2;
51         return combine(query(v * 2, tl, tm, l, min(r,
52             tm)),
53             query(v * 2 + 1, tm + 1, tr,
54                 max(l, tm + 1), r));
55     }
56 public:
57     SegmentTree(vector<T>& arr) {
58         n = arr.size();
59         tree.resize(4 * n);
60         build(arr, 1, 0, n - 1);
61     }
62     void update(int pos, T new_val) {
63         update(1, 0, n - 1, pos, new_val);
64     }
65     T query(int l, int r) {
66         return query(1, 0, n - 1, l, r);
67     }
68 };

```